

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Магистерская программа «Искусственный интеллект»

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

Исследовательская выпускная квалификационная работа на тему:
Обучение рассуждению без верификаторов для генерации юмора

Выполнил студент:

2 курса магистерской программы

ФИО студента

Принял руководитель ВКР:

ФИО научного руководителя

Должность, ученая степень (при наличии)

Факультет компьютерных наук НИУ ВШЭ

Москва 2026

Оглавление

Аннотация	4
Ключевые слова	5
Введение	6
1 Обзор литературы	9
1.1 Вычислительный юмор как исследовательская проблема	9
1.2 Генерация юмора большими языковыми моделями	11
1.3 Проблема оценки юмора	12
1.4 Рассуждение и творческий поиск в генерации юмора	13
1.5 Ограничения стандартного постобучения	14
1.6 Обучение без внешнего верификатора	15
1.7 Позиционирование данной работы	16
2 Методология	18
2.1 Границы исследования и статус реализации	18
2.2 Построение набора данных	19
2.3 Плотный индекс векторных представлений	19
2.4 Извлечение ключевых слов как поиск слабых запросов	20
2.5 Построение эталонных ответов через группировку ключевых слов и поиск	21
2.5.1 Группировка ключевых слов и формирование запросов . .	21
2.5.2 Поиск эталонных ответов с помощью индекса векторных представлений	22
2.5.3 Удаление дублей и разбиение выборки	23
2.6 Генерация кандидатов	24
2.7 Оценка кандидатов	24

2.7.1	Попарное сравнение	24
2.7.2	Агрегация таблицы результатов	25
2.8	Теоретическая рамка обучения MRVF	25
2.8.1	Обозначения	26
2.8.2	RL с верифицируемыми наградами	27
2.8.3	VeriFree	29
2.8.4	С несколькими эталонными ответами VeriFree	31
2.8.5	Неполные наблюдаемые эталонные ответы	33
2.8.6	Предлагаемый цикл обучения	36
2.8.7	Замечание о использовании базового уровня	38
3	Результаты	40
3.1	Текущие результаты	40
3.2	Планируемая процедура обучения	40
3.3	Планируемые абляции	42
	Заключение	43
	Список литературы	45
A	Реализованные шаблоны запросов	49
A.1	Запрос для генерации эталонных ответов	49
A.2	Инструкции для запросов для векторного поиска	49
A.3	Оценочный запрос	50
B	Дополнительные замечания о программной архитектуре	51

Аннотация

Ключевые слова

Введение

Генерация юмора является удобным способом изучения ограничений современных методов постобучения языковых моделей. В таких областях, как математика или программирование, сгенерированный ответ часто можно проверить внешней процедурой: решение либо совпадает с правильным ответом, либо программа проходит тесты или не проходит их. Юмор устроен иначе. Шутка может быть грамматически правильной, связной и соответствующей теме, но при этом не производить нужного эффекта. При этом проблема не сводится только к субъективности юмора. Один и тот же запрос может допускать множество разных удачных шуток, тогда как наблюдаемый набор данных обычно содержит лишь небольшую их часть. Поэтому юмор является не только сложной задачей генерации, но и полезным примером для изучения рассуждения и обучения с подкреплением в невалидируемых доменах [14, 11].

Эта сложность стала особенно важной после появления больших языковых моделей. Ранние системы вычислительного юмора были узкими, но интерпретируемыми: они опирались на шаблоны, лексические ресурсы или конкретные механизмы каламбуров [15, 1]. Современные языковые модели обладают гораздо более широким покрытием, однако недавние работы показывают, что беглость текста не равна сильной юмористической компетентности. Крупные оценки по-прежнему помещают модельные системы ниже сильных человеческих результатов, а исследования каламбуров и понимания шуток показывают, что модели часто надежнее воспроизводят поверхностную форму юмора, чем его внутренний механизм [24, 21, 4, 13]. Иными словами, юмор выявляет знакомую слабость современных моделей: они хорошо продолжают вероятный текст, но хуже выполняют открытый поиск по множеству правдоподобных и неожиданных продолжений.

Один из ответов на эту слабость состоит в явном моделировании процесса рассуждения. Современные системы генерации юмора все чаще используют поэтапное построение подсказок, творческий поиск или декомпозицию на основе

теории, а не одношаговое завершение текста. Эти подходы мотивированы простым наблюдением: хорошие шутки часто возникают не через выбор первого вероятного продолжения, а через исследование ассоциаций, переосмыслений, контрастов и форм панчлайна [2, 25, 17, 10, 23]. Результаты выглядят многообещающе, но имеют ограничение. В большинстве таких систем структура рассуждения задается вручную: исследователь определяет число этапов, тип промежуточных представлений и порядок действий модели.

Параллельное направление в постобучении предлагает другую возможность. В верифицируемых доменах обучение с подкреплением недавно использовалось для извлечения развернутого поведения рассуждения напрямую, а не только через ручное построение подсказок или фиксированные процессы [12, 5]. Методы без внешнего верификатора, такие как VeriFree и RLPR, дополнительно показывают, что часть этой логики может быть перенесена за пределы математики и программирования, если заменить внешнего верификатора внутренними вероятностными целями [26, 22]. Однако для юмора такие формулировки сразу сталкиваются с несоответствием самой природе задачи. Они построены вокруг предположения об одной эталонной ссылке, тогда как юмор естественно является задачей с несколькими эталонными ответами: для запроса вроде “write a joke about a frog” существует много приемлемых ответов, и любой набор данных фиксирует только некоторые из них.

Данная работа мотивирована именно этим несоответствием. Ее центральное утверждение состоит в том, что генерацию юмора следует формулировать не как точную имитацию наблюдаемой шутки, а как распределение вероятностной массы по множеству приемлемых шуток при заданном запросе. Этот сдвиг важен и теоретически, и практически. Теоретически он меняет способ определения обучения без внешнего верификатора с подкреплением в творческом домене. Практически он меняет требования к данным: вместо изолированных пар запрос–ответ обучение требует наборов эталонных ответов, связанных с одним запросом и содержащих несколько правдоподобных шуток.

Работа поэтому состоит из двух связанных частей. Первая часть является методологической и теоретической: в ней формулируется расширение обучения с подкреплением без внешнего верификатора на случай нескольких эталонных ответов для генерации юмора и анализируются последствия неполноты наблюдаемых эталонных наборов. Вторая часть является реализационной: она строит

процесс данных, необходимый для практического применения такой цели. В текущем репозитории этот процесс уже включает сбор корпуса, генерацию плотных векторных представлений, извлечение ключевых слов, построение запросы, поиск эталонных ответов, генерацию кандидатов и автоматическую оценку. Отсутствующим этапом остается сам цикл обучения. Соответственно, данную работу следует читать как предварительное исследование в точном смысле этого термина: она задает формальную цель и программную инфраструктуру, необходимую для последующего обучающего эксперимента.

Источником данных является объединенный корпус примерно из 664k шуток, собранный из коллекций Short Jokes и rJokes. Полный корпус используется как пул поиска, тогда как его ключевых слов достаточно для построения обучающего набора данных с несколькими эталонными ответами. Целевая экспериментальная постановка является узким и контролируемым: модели Qwen3-1.7B и Qwen3-4B в режимах instruct и thinking будут сравниваться с вариантом thinking, обученным с помощью MRVF; обучение планируется на NVIDIA H200 GPU с использованием модифицированного `trl` и `vLLM` для генерации траекторий и вывода базовых моделей.

Глава 1

Обзор литературы

1.1 Вычислительный юмор как исследовательская проблема

Юмор является одной из наиболее трудных задач для обработки естественного языка, поскольку успешная шутка определяется не только грамматической правильностью или тематической релевантностью. Она обычно требует нарушения ожиданий, использования общего знания, выбора подходящей формы, учета аудитории и точного соотношения между подготовкой и неожиданным завершением. Поэтому генерация юмора занимает промежуточное положение между задачами языкового моделирования, творческого письма и прагматического анализа коммуникации.

Ранние исследования вычислительного юмора уже подчёркивали, что юмор плохо сводится к одному формальному правилу. Ritchie [15] описывает вычислительный юмор как область, в которой частные механизмы могут быть формализованы, но общая способность понимать и порождать юмор требует знания о языке, мире и намерениях участников коммуникации. Обзор Amin и Burghardt [1] показывает, что классические системы генерации юмора часто строились вокруг узких механизмов: шаблонов, лексических ресурсов, фонетических замен, каламбуров или заранее заданных форм шутки. Такие системы были интерпретируемыми и позволяли контролировать юмористический механизм, но плохо переносились на открытые темы и разнообразные запросы.

Современные обзоры приходят к сходному выводу уже в контексте больших языковых моделей. Loakman, Thorne и Lin [14] отмечают, что генерация и объяснение юмора остаются недостаточно изученными по сравнению с более стандартными задачами обработки естественного языка, особенно если выхо-

доть за пределы каламбуров. Lemmens и De Marez [11] также подчёркивают, что несмотря на рост возможностей нейросетевых моделей, юмор остаётся областью с неоднозначными результатами: модели способны воспроизводить внешние признаки юмористического текста, но это не означает устойчивого понимания юмористических механизмов.

Каламбуры являются удобным примером этой проблемы. С одной стороны, они дают относительно чёткий объект анализа: в них можно изучать неоднозначность, фонетическое сходство и переключение смысла. Так, Kao, Levy и Goodman [9] формализуют юмор в каламбурах через сочетание неоднозначности и отличительности, показывая, что эти факторы связаны с человеческими оценками смешного. С другой стороны, успех на каламбурах не переносится автоматически на более широкие формы юмора: короткие нарративные шутки, тематические шутки, ситуационный юмор или социально обусловленные ответы. Современные исследования больших языковых моделей подтверждают это ограничение. Ху и др. [21] показывают, что модели часто демонстрируют «ленивую» генерацию каламбуров: вместо компактного совмещения двух значений они явно называют оба связанных понятия или воспроизводят поверхностный шаблон игры слов. Cosschieri и др. [4] также показывают, что даже сильные языковые модели испытывают трудности с обобщением в задачах понимания и генерации юмора, когда требуется не просто распознать знакомую форму, а применить юмористический механизм в новой ситуации.

Для данной работы из этого следует два вывода. Во-первых, генерацию юмора нельзя рассматривать как обычную задачу продолжения текста: беглость и тематическая релевантность являются необходимыми, но недостаточными условиями. Во-вторых, юмор является задачей с множеством допустимых ответов. Для одного и того же запроса может существовать большое число хороших шуток, основанных на разных ассоциациях и разных механизмах неожиданности. Это делает постановку с единственным эталонным ответом методологически слабой для обучения и оценки юмористической генерации.

1.2 Генерация юмора большими языковыми моделями

Появление больших языковых моделей изменило характер исследований генерации юмора. В отличие от шаблонных систем, такие модели способны отвечать на широкий спектр запросов и порождать тексты, которые по форме похожи на человеческие шутки. Однако литература показывает, что это расширение охвата не устраняет центральные трудности области.

Крупные корпуса юмора, такие как rJokes, важны как источники разнообразных реальных шуток [19]. Они позволяют изучать тематическое разнообразие, повторяющиеся формы и статистические свойства юмористических текстов. Однако такие корпуса обычно не строятся как наборы данных для многореференсной генерации. В них нет явного соответствия между одним запросом и множеством допустимых ответов. Поэтому прямое дообучение модели на корпусе шуток может научить её стилю и распространённым формам, но не обязательно научит выбирать неожиданный и уместный юмористический ход для нового запроса.

Ограничения современных моделей хорошо видны в задачах, где существует сильная человеческая оценка. Zhang и др. [24] представляют крупный ресурс для генерации подписей к карикатурам, содержащий более 250 миллионов человеческих оценок и более 2.2 миллиона подписей. Эта работа особенно важна, поскольку рассматривает юмор как творческую задачу предпочтений, а не как бинарную классификацию. Авторы показывают, что даже сильные модели и методы дообучения по предпочтениям остаются ограниченными на творческой задаче: лучшие системы всё ещё уступают сильным человеческим участникам.

Похожие выводы появляются и в более специализированных исследованиях. Loakman, Thorne и Lin [13] анализируют понимание юмора от традиционных каламбуров до тематических шуток и показывают, что современные модели не одинаково хорошо справляются с разными типами юмора. Sakabe и др. [16] рассматривают японскую импровизационную форму Oogiri и оценивают ответы по нескольким измерениям: новизне, ясности, релевантности, интеллектуальности, эмпатии и общей смешности. Их результаты показывают, что языковые модели могут достигать уровня между низкими и средними человеческими результатами.

ми, но заметно отличаются от людей по тому, какие качества ответа они считают важными.

Эти работы показывают, что прогресс больших языковых моделей не делает задачу генерации юмора решённой. Скорее, он меняет исследовательский вопрос. Если раньше основная трудность состояла в том, чтобы вообще получить грамматически и структурно похожий на шутку текст, то теперь важно отличать поверхностную имитацию от устойчивого творческого обобщения. Для данной работы это особенно существенно: оптимизируемая модель не должна просто запоминать распространённые шутки или воспроизводить корпусные шаблоны, а должна использовать эталонные шутки как слабый сигнал для обучения распределения творческих траекторий.

1.3 Проблема оценки юмора

Оценка юмора является отдельной методологической трудностью. В отличие от задач с объективно проверяемым ответом, смешность зависит от аудитории, контекста, культурных ожиданий, формы подачи и конкретного измерения качества. Поэтому простая формулировка «ответ смешной или нет» часто оказывается слишком грубой.

Корпус rJokes уже демонстрирует эту проблему: реальные шутки разнообразны по теме, форме и качеству, а их восприятие зависит от сообщества, в котором они были опубликованы [19]. Более поздние работы делают этот вывод сильнее. Sakabe и др. [16] показывают, что оценка юмора должна учитывать несколько измерений, а не только общую смешность. Их анализ также показывает расхождение между человеческими и модельными критериями оценки: люди и языковые модели могут придавать разный вес новизне, эмпатии и релевантности.

Winters и Stockt [20] предлагают другой важный взгляд, сравнивая генерацию юмора в условиях, близких к импровизационной комедии. Такая постановка подчёркивает, что юмор не является только свойством изолированного текста. На оценку влияет ситуация, скорость ответа и ожидания аудитории. Следовательно, результаты статической текстовой оценки не следует напрямую отождествлять с качеством юмора в живом взаимодействии.

В то же время полностью отказаться от автоматической оценки в вычис-

лительных экспериментах трудно. Человеческая оценка дорога, шумна и плохо масштабируется. Поэтому современные работы используют языковые модели как вспомогательных оценщиков или строят более структурированные рубрики. Hashemi и др. [8] предлагают многомерный и калиброванный подход к автоматической оценке текстов, показывая, что оценивание с помощью языковых моделей требует явной структуры и контроля. Gunjal и др. [7] рассматривают рубрики как источник награды для обучения за пределами строго проверяемых доменов. Эти подходы важны для данной работы, поскольку юмор также относится к областям, где внешняя награда является неполной и шумной.

Тем не менее модель-оценщик не является прозрачной заменой человеческого восприятия. В данной работе автоматическая попарная оценка используется как воспроизводимый инструмент промежуточного сравнения моделей, а не как окончательное измерение человеческой смешности. Это ограничение принципиально: даже если модель выигрывает в попарной оценке, такой результат следует интерпретировать как улучшение относительно выбранного протокола оценки, а не как доказательство универсального юмористического качества.

1.4 Рассуждение и творческий поиск в генерации юмора

Один из важных сдвигов последних лет состоит в переходе от мгновенной генерации ответа к явному моделированию промежуточного рассуждения. В широком контексте обработки естественного языка Wei и др. [18] показывают, что подсказки с цепочкой рассуждения могут существенно менять поведение больших языковых моделей на задачах, требующих многошагового вывода. Для юмора этот результат нельзя переносить напрямую: юмористическое мышление не всегда является линейным доказательством. Однако сама идея промежуточного поиска оказывается релевантной.

Chen, Shi и Si [2] рассматривают генерацию юмора как процесс, который можно направлять пошаговыми инструкциями, основанными на принципах комедийного письма. Такой подход показывает, что явная организация творческого процесса может улучшать ответы по сравнению с простой генерацией по запросу. Tikhonov и Shtykovskiy [17] развивают похожую идею для коротких

шуток, реконструируя процесс их создания как многошаговый механизм. В их постановке модель не просто выдаёт финальную шутку, а проходит через промежуточные этапы, связанные с поиском предпосылки, неожиданного поворота и окончательной формулировки.

Другие работы подчёркивают, что творческие задачи требуют не только последовательного рассуждения, но и ассоциативных скачков. Zhong и др. [25] предлагают парадигму Leap-of-Thought для генерации юмора в формате Oogiri. В отличие от стандартной цепочки рассуждения, такая схема акцентирует внимание на неожиданных ассоциациях между отдалёнными понятиями. Kim и Chilton [10] описывают генерацию юмора как сочетание когнитивных, социальных и творческих навыков, а Zhang и др. [23] предлагают многоэтапную теоретически направляемую схему для интерпретируемой мультимодальной генерации юмора.

Эти исследования важны для данной работы по двум причинам. Во-первых, они подтверждают, что промежуточные представления и творческий поиск могут быть полезны для генерации юмора. Во-вторых, большинство таких подходов задаёт структуру рассуждения вручную: исследователь заранее определяет этапы, их порядок и типы промежуточных объектов. В данной работе используется другая перспектива. Траектория рассуждения рассматривается как случайная переменная, которую модель может порождать сама. Обучение должно не только повысить вероятность конкретных эталонных шуток, но и изменить распределение траекторий так, чтобы они чаще приводили к правдоподобным юмористическим ответам.

1.5 Ограничения стандартного постобучения

Наиболее прямой способ улучшить генерацию юмора состоит в дообучении модели с учителем на корпусе шуток. Однако такая постановка плохо соответствует структуре задачи. Для одного запроса существует много допустимых шуток, и наблюдаемая в корпусе шутка является только одним из возможных вариантов. Если модель обучается воспроизводить единственный ответ, она может улучшать стилистическую похожесть на корпус, но не обязательно улучшать способность к творческому обобщению.

Общая литература по постобучению усиливает это опасение. Chu и др. [3]

противопоставляют дообучение с учителем и обучение с подкреплением, показывая, что первое может сильнее запоминать обучающие шаблоны, тогда как второе при удачной постановке лучше переносится на новые варианты задач. Для юмора это различие особенно важно: запоминание частотных форм шуток не равно способности построить новый неожиданный панчлайн.

Оптимизация по предпочтениям и обучение с подкреплением по модели награды также имеют ограничения. Gao, Schulman, Hilton и др. [6] показывают, что чрезмерная оптимизация несовершенной модели награды может ухудшать истинное качество. В творческих задачах этот риск особенно велик, потому что модель награды или модель-оценщик неизбежно отражает неполные и шумные критерии. Работа Zhang и др. [24] показывает ограничения RLHF- и DPO-подобных методов для юмористических подписей к карикатурам, а результаты Sakabe и др. [16] и Winters и Stockt [20] подчёркивают расхождение между модельными и человеческими оценками юмора.

Следовательно, стандартные методы постобучения не решают задачу полностью. Дообучение с учителем может усиливать имитацию корпуса, а обучение по внешней модели награды может оптимизировать неполный или смещённый критерий. Для данной работы это мотивирует поиск промежуточной постановки: использовать корпус эталонных шуток как слабый сигнал качества, но не обучаться на нём как на единственно правильных ответах и не вводить отдельную модель награды.

1.6 Обучение без внешнего верификатора

Недавние успехи обучения с подкреплением для рассуждающих языковых моделей во многом связаны с доменами, где доступна объективная проверка ответа. В математике и программировании можно задать верификатор, который проверяет правильность результата. Lightman и др. [12] показывают пользу процессной разметки для математических задач, а DeepSeek-AI [5] демонстрируют, что обучение с подкреплением может вызывать длинные траектории рассуждения даже без заранее написанных человеком цепочек рассуждения. Однако такие результаты опираются на возможность проверить ответ, что плохо переносится на творческие задачи.

Подходы без внешнего верификатора пытаются заменить явную проверку

внутренними вероятностями самой модели. Zhou и др. [26] предлагают VeriFree: вместо того чтобы генерировать ответ и проверять его внешним модулем, метод максимизирует вероятность эталонного ответа при условии запроса и сгенерированной траектории рассуждения. В такой постановке вероятность эталонного ответа одновременно играет роль сигнала награды для траектории рассуждения и источника прямого градиента для увеличения правдоподобия эталонного ответа.

Близкую идею развивают Yu и др. [22] в методе RLPR. Они используют внутренние вероятностные оценки модели как сигнал награды и подчёркивают необходимость стабилизации такого сигнала, поскольку вероятности длинных ответов имеют высокую дисперсию и могут становиться численно неудобными. Этот вывод особенно важен для генерации юмора. В задачах с коротким ответом вероятность эталонной последовательности ещё может быть практическим сигналом. Для полной шутки, состоящей из десятков токенов, произведение токеновых вероятностей быстро становится очень малым и начинает сильно зависеть от длины ответа.

Ограничение методов VeriFree и RLPR для данной работы состоит в том, что они в первую очередь формулируются вокруг одного эталонного ответа или вокруг задач, где эталонный ответ можно считать достаточно определённым. Юмор устроен иначе. Один запрос может иметь множество хороших ответов, а наблюдаемое множество шуток является неполным и смещённым подмножеством возможных удачных вариантов. Поэтому прямое применение одноэталонной цели может ошибочно наказывать хорошие, но не наблюдавшиеся шутки, и чрезмерно усиливать отдельные формулировки из корпуса.

1.7 Позиционирование данной работы

Данная работа находится на пересечении трёх направлений: генерации юмора, обучения рассуждающих языковых моделей и обучения без внешнего верификатора. Литература по вычислительному юмору показывает, что задача является многомерной, контекстно зависимой и плохо сводится к единственной метрике [15, 1, 14, 11]. Работы по генерации юмора с рассуждением показывают, что промежуточный творческий поиск может улучшать ответы, но часто задают структуру этого поиска вручную [2, 25, 17, 10, 23]. Работы по обучению

без внешнего верификатора показывают, что вероятность эталонного ответа может служить внутренним сигналом обучения, но их исходная постановка плохо учитывает многореференсность и неполноту творческих задач [26, 22].

Основной исследовательский пробел состоит в том, что существующие методы либо рассматривают юмор как задачу генерации по вручную заданному сценарию рассуждения, либо рассматривают обучение рассуждений в доменах с объективной проверкой или одним эталонным ответом. В данной работе предлагается адаптировать идею обучения без внешнего верификатора к юмору как к частично наблюдаемой многореференсной задаче. Вместо единственного эталонного ответа используется множество эталонных шуток, найденных в корпусе по близости запроса. Вместо сырой вероятности полной шутки используется нормированная логарифмическая масса эталонных ответов, что уменьшает численные проблемы и смещение в пользу коротких последовательностей. Траектория рассуждения при этом остаётся порождаемой моделью, а не задаётся фиксированным шаблоном.

Такое позиционирование определяет вклад работы. Она не утверждает, что автоматическая оценка юмора полностью заменяет человеческую, и не предполагает, что наблюдаемые эталонные шутки исчерпывают все хорошие ответы. Напротив, работа рассматривает эти ограничения как часть постановки задачи. Цель состоит в разработке и экспериментальной проверке программного конвейера, который позволяет изучать, можно ли использовать многореференсную логарифмическую цель для обучения рассуждающей генерации юмора без отдельной модели награды.

Глава 2

Методология

2.1 Границы исследования и статус реализации

Методология данной предварительной дипломной работы имеет два уровня. Первый уровень — реализованный процесс подготовки данных и оценки, уже присутствующий в репозитории. Второй уровень — предлагаемая обучающая цель, а именно с несколькими эталонными ответами без внешнего верификатора (MRVF) обучение с подкреплением, которая математически сформулирована, но еще не полностью реализована в коде. Это различие важно для границ настоящей работы. Репозиторий уже содержит процессы для построения корпуса, генерации векторных представлений, извлечения ключевых слов, поиска эталонных ответов, генерации кандидатов и автоматической попарной оценки. Однако сам этап обучения остается будущей работой. Поэтому данная глава различает то, что уже операционально в кодовой базе, и то, что предлагается как следующий этап проекта.

На высоком уровне реализованная система строит слабо размеченный набор данных с несколькими эталонными ответами из сырых наборов данных шуток. Сначала шутки объединяются в единый корпус, затем кодируются моделью плотного векторного поиска и сводятся к небольшому набору ключевых слов. Эти ключевые слова расширяются в группы запросов на основе n -грамм, которые затем используются для поиска семантически близких шуток в индексе векторных представлений. Полученные эталонные ответы имеют два непосредственных применения. Во-первых, они дают данные для генерации базовой моделью и автоматической оценки. Во-вторых, они задают обучающую и валидационную выборки, необходимые для предлагаемой целевой функции обучения MRVF.

2.2 Построение набора данных

Набор данных объединяется из двух публичных коллекций шуток: наборов данных Short Jokes и rJokes. В репозитории этот этап реализован в `src/pipelines/jokes.py`. Конвейер скачивает сырые файлы из зафиксированных версий источников, преобразует их в формат Parquet и сохраняет метаданные источника для каждой шутки. Для каждой строки сохраняются поля `id`, `text`, `source_name`, `source_filename` и `source_id`. После отдельной предобработки двух корпусов они конкатенируются и получают новый глобальный целочисленный идентификатор. Итоговый набор данных содержит примерно 664k объектов.

Такой дизайн намеренно является консервативным. Процесс не выполняет агрессивную нормализацию текста, кроме удаления пустых записей и стандартизации формата хранения. Для юмора такая осторожность желательна, поскольку кажущиеся мелкими текстовыми детали, такие как пунктуация, формулировка или переносы строк, могут влиять на комический эффект. Сохранение метаданных источника также полезно для последующего анализа доменных различий, поиска дубликатов и возможных смещений, связанных с источником.

2.3 Плотный индекс векторных представлений

Следующий этап кодирует каждую шутку в плотном семантическом векторном пространстве. Он реализован в `src/pipelines/embeddings.py`. Текущая конфигурация по умолчанию использует модель Qwen3-Embedding-8B с размерностью векторного представления 1024, уменьшенной с исходных 4096 с помощью Matryoshka Representation Learning. Векторные представления вычисляются асинхронно пакетами, записываются в фрагменты Parquet и хранятся как пары (i, e_i) , где i — глобальный идентификатор шутки, а $e_i \in \mathbb{R}^{1024}$ — соответствующее векторное представление.

Роль этого этапа двоякая. Во-первых, векторные представления шуток затем используются для оценки ключевых слов-кандидатов по семантической релевантности. Во-вторых, они задают векторный индекс, используемый для поиска наборов шуток с несколькими эталонными ответами. Иными словами, пространство векторных представлений является общим представлением, от ко-

торого зависят и извлечение ключевых слов, и поиск эталонных ответов.

2.4 Извлечение ключевых слов как поиск слабых запросов

Предполагается, что каждая шутка имеет набор слабых запросов, на которые она правдоподобно отвечает. Например, “write a joke about a chicken” рассматривается как допустимый запрос для шутки “why did the chicken cross the road”. Этап извлечения ключевых слов направлен на построение слабых запросов на основе ключевых слов шуток. Он реализован в `src/pipelines/keywords.py`.

Для каждой шутки y_i процесс сначала генерирует пул n -грамм-кандидатов с помощью анализатора `CountVectorizer`. В конфигурации по умолчанию этот анализатор использует только униграммы, удаляет английские стоп-слова и оставляет не более 64 терминов-кандидатов на шутку. Пусть полученный набор кандидатов равен

$$C(y_i) = \{c_{i1}, \dots, c_{im}\}.$$

Затем каждый термин-кандидат кодируется той же моделью векторных представлений, что и шутки. Если $e(y_i)$ обозначает векторное представление шутки, а $e(c)$ обозначает векторное представление ключевого слова-кандидата, то оценка релевантности вычисляется через косинусное сходство:

$$s(c, y_i) = \frac{e(c)^\top e(y_i)}{\|e(c)\| \|e(y_i)\|}.$$

Выбор только наиболее похожих кандидатов часто давал бы избыточные ключевые слова. Для уменьшения избыточности процесс применяет `maximal marginal relevance (MMR)`, выбирая несколько ключевых слов с балансом между релевантностью и разнообразием. В конфигурации по умолчанию коэффициент разнообразия равен 0.7, а число сохраняемых ключевых слов равно 3. Поэтому выход этого этапа — компактное представление в виде ключевых слов

$$K(y_i) = \{k_{i1}, k_{i2}, k_{i3}\},$$

которое может содержать меньше трех элементов, если пул кандидатов мал.

Методологически эти ключевые слова не следует интерпретировать как полноценные скрытые запросы. Это более слабые объекты: n -граммы, которые захватывают тематические или лексические якоря шуток. Это прагматическое приближение. Оно не решает восстановление запроса по шутке полностью, но дает удобное представление, из которого можно построить несколько вариантов запроса.

2.5 Построение эталонных ответов через группировку ключевых слов и поиск

2.5.1 Группировка ключевых слов и формирование запросов

Этап построения эталонных ответов реализован в `src/pipelines/references`. Для каждого набора ключевых слов $K(y_i)$ процесс перечисляет все уникальные комбинации ключевых слов, размер которых лежит между `min_keywords` и `max_keywords`. В конфигурации по умолчанию эти значения равны 1 и 2, поэтому процесс строит все однословные и двухсловные группы. Если

$$K(y_i) = \{k_1, k_2, k_3\},$$

то сгенерированные группы равны

$$\{k_1\}, \{k_2\}, \{k_3\}, \{k_1, k_2\}, \{k_1, k_3\}, \{k_2, k_3\}.$$

Каждая группа преобразуется в запрос на естественном языке с помощью шаблона

`Write a joke using the following keyword(s): {keywords}`

Этот запрос служит двум целям: это текстовая форма, которая затем подается генеративным моделям, и это же объект, который кодируется как запрос для поиска эталонных ответов.

Такой дизайн создает контролируемый, но разнообразный спектр запросов. Запросы с одним ключевым словом являются широкими, тематическими и

часто допускают много возможных шуток. Запросы с двумя ключевыми словами более ограничены и обычно извлекают более точные кандидаты. Текущая реализация останавливается на двух ключевых слов, потому что более крупные группы быстро становятся слишком специфичными, уменьшая число правдоподобных различных эталонных ответов на запрос.

2.5.2 Поиск эталонных ответов с помощью индекса векторных представлений

После формирования строк запросов процесс кодирует их как запросы и извлекает близкие шутки из индекса FAISS, построенного по полной коллекции векторных представлений шуток. Индекс поиска является инвертированным файловым индексом (IVF). Перед индексацией и поиском векторы нормализуются, так что скалярное произведение соответствует косинусному сходству. В конфигурации по умолчанию основные параметры таковы:

- модель векторных представлений: Qwen3-Embedding-8B;
- размерность: 1024;
- `faiss_nlist`: 4096;
- `faiss_nprobe`: 32;
- `top_k`: 5;
- `oversample`: 10;
- `min_similarity`: 0.5.

Для сформированного запроса x пусть $q(x)$ — его нормализованное векторное представление, а e_j — нормализованное векторное представление шулки y_j . Тогда оценка поиска равна

$$s(x, y_j) = q(x)^\top e_j.$$

Процедура поиска извлекает больше, чем k кандидатов, фильтрует те, чье сходство ниже порога, и оставляет k лучших оставшихся объектов. Соответствующе

щие тексты затем используются как наблюдаемый набор эталонных ответов:

$$Y^*(x) = \{y_1, \dots, y_n\}, \quad n \leq 5.$$

На практике поиск эталонных ответов дает около 4 эталонных ответов на группу ключевых слов после удаления дублей и разбиения выборки. Целевой объем выборки для обучения равен 10 – 20k эталонных ответов, поэтому достаточно сгенерировать 3 – 5k групп ключевых слов.

Этот этап реализует центральную идею работы: запрос связывается с несколькими шутками-кандидатами в качестве эталонных ответов, а не с одним истинным ответом. Важно, что найденный набор не является исчерпывающе размеченным множеством всех хороших шуток для x . Это приближение, полученное поиском такого множества. Это существенно и эмпирически, и теоретически. Эмпирически найденные эталонные ответы могут содержать шум или пропускать валидные альтернативы. Теоретически неполнота $Y^*(x)$ становится ключевой частью дальнейшего анализа MRVF, а не второстепенным неудобством.

2.5.3 Удаление дублей и разбиение выборки

После поиска итоговый набор данных содержит строки вида

$$(x_i, Y^*(x_i), S(x_i)),$$

где x_i — запрос на основе ключевых слов, $Y^*(x_i)$ — найденный набор эталонных ответов, а $S(x_i)$ — вектор оценок поиска. Разные исходные шутки могут генерировать одинаковые группы ключевых слов, поэтому репозиторий дедуплицирует набор данных по полю **keywords**, сохраняет первое вхождение и заново назначает идентификаторы строк.

Дедуплицированный набор данных затем разбивается на обучающую, валидационную и тестовую части. Текущая реализация использует двухэтапное случайное разбиение с `test_fraction=0.1`, `validation_fraction=0.1` и фиксированным зерном случайности. Отдельные фрагменты Parquet записываются для полного набора данных и для каждой части выборки.

2.6 Генерация кандидатов

Этот этап поддерживает генерацию кандидатов на основе набора эталонных ответов и реализован в `src/pipelines/candidates.py`. Для выбранной модели и части набора данных процесс проходит по запросам и вызывает интерфейс генерации ответов, совместимый с OpenAI. Используется тот же шаблон запроса, что и при построении эталонных ответов:

```
Write a joke using the following keyword(s): {keywords}
```

В конфигурации по умолчанию генерация кандидатов использует температуру 1.0 и допускает до 128 токенов завершения. Важно сохранять шутки короткими, чтобы упростить оценку как с помощью языковой модели-оценщика, так и в условиях человеческой разметки.

Схема выхода содержит четыре поля: идентификатор строки, группа ключевых слов, название модели и сгенерированный текст. Файлы с кандидатами сохраняются в структуре каталогов по модели и части выборки. Такая организация важна для последующей оценки, поскольку позволяет выравнивать ответы нескольких моделей по одному базовому идентификатору и запросу.

Методологически этот этап генерации кандидатов следует понимать как слой контрольного набора. Он создает артефакты, необходимые для сравнения базовых моделей: предобученная базовая модель, версий в режимах `instruct` и `thinking`, а также будущих обученных контрольных точек.

2.7 Оценка кандидатов

2.7.1 Попарное сравнение

Автоматическая оценка реализована в `src/pipelines/evaluation.py`. Процесс сначала конкатенирует наборы данных с кандидатами от нескольких моделей. Затем он группирует кандидаты по идентификатору эталонного набора и название модели и строит попарные сравнения для всех пар моделей, которые произвели ответы для одного запроса. Порядок левого и правого ответа рандомизируется с фиксированным зерном случайности, чтобы уменьшить смещение из-за позиции ответа.

Оценочный запрос намеренно прост: он содержит общий запрос на шутку, а также левый и правый кандидаты. Системная инструкция просит разметчик действовать как оценщик качества юмора, выбрать, какая шутка смешнее для широкой аудитории, избегать ничьих и возвращать строгий JSON. В конфигурации по умолчанию модель-оценщик работает при температуре 0.

Этот слой оценки захватывает только сравнительную смешность при общем запросе. Он пока не реализует многомерную оценку, обсуждаемую в обзоре литературы: отдельные суждения по рубрикам для соответствия запросу, новизны, похожесть на человеческий ответ и смешность; также он пока не включает человеческую разметку. Эти этапы остаются будущей работой, поскольку зависят от завершенной процедуры обучения.

2.7.2 Агрегация таблицы результатов

Попарные суждения агрегируются в рейтинг с числом побед, числом поражений, числом сравнений, долей побед и оценками Bradley-Terry. Пусть $\mathcal{M}$ обозначает множество оцениваемых моделей. Для каждой оцениваемой пары (m_a, m_b) оценщик записывает бинарную победу для одной стороны. Затем процесс оценивает параметры Bradley-Terry и сообщает их логарифмы как значения `bt_score`.

Такая агрегация разумна для юмора, потому что относительные суждения часто стабильнее абсолютных оценок. Судье обычно легче решить, какая из двух шуток смешнее, чем дать хорошо калиброванную числовую оценку. В то же время эта процедура наследует ограничения модель-оценщик. Поскольку суждения о юморе языковых моделей могут расходиться с человеческими суждениями, текущий рейтинг следует интерпретировать как быстрый и дешевую приближенную меру для промежуточной валидации, а не как окончательную меру качества юмора.

2.8 Теоретическая рамка обучения MRVF

Реализованный выше процесс строит наборы эталонных ответов, обусловленные запросом; данный раздел определяет, как такие множества входят в целевую функцию обучения с подкреплением, ориентированного на рассуждение. Центральное утверждение состоит в том, что генерация юмора плохо

согласуется с предположениями об одном эталонном ответе, используемыми в стандартном основанном на верификаторе или без внешнего верификатора обучении с подкреплением для рассуждения. Слабый запрос может допускать множество приемлемых шуток, и репозиторий уже делает это конкретным, извлекая несколько эталонных ответов для одного запроса на основе ключевых слов. Ниже формализуется оптимизация по такой разметке в виде множества допустимых ответов.

2.8.1 Обозначения

Пусть $\mathcal{X}$, $\mathcal{Z}$ и $\mathcal{Y}$ обозначают пространства запросов, траекторий рассуждения и итоговых ответов соответственно. Для запроса $x \in \mathcal{X}$:

- $z \in \mathcal{Z}$ обозначает траекторию рассуждения,
- $y \in \mathcal{Y}$ обозначает итоговую сгенерированную шутку,
- $y^* \in \mathcal{Y}$ обозначает один эталонный ответ,
- $Y^*(x) \subset \mathcal{Y}$ обозначает *наблюдаемый* набор допустимых эталонных ответов, связанный с запросом x ,
- π_θ обозначает стратегия, то есть условную вероятностную модель над текстом, параметризованную θ .

Более явно,

$$\pi_\theta(z, y \mid x) = P_\theta(Z = z, Y = y \mid X = x).$$

Стратегия факторизуется как

$$\pi_\theta(z, y \mid x) = \pi_\theta(z \mid x) \pi_\theta(y \mid x, z).$$

Когда модель явно выводит токены рассуждения, z можно отождествить с текстом внутри специального сегмента `<think>`, а y — с итоговым ответом, который следует после него. Когда рассуждение явно не экспонируется в выходной поток, ту же факторизацию можно понимать как концептуальное разложение на выбранную промежуточную траекторию z и итоговый ответ, обусловленный

этой траекторией. Преимущество такой нотации в том, что она разделяет две роли модели: выбор пути рассуждения и распределение вероятностной массы по ответам при данном пути.

Для краткости целевая функция оптимизации ниже записывается через наблюдаемый набор $Y^*(x)$. При обсуждении неполноты будет полезно представлять более крупное скрытое множество $Y_{\text{true}}(x) \supseteq Y^*(x)$, содержащее все действительно приемлемые шутки для запроса x , хотя это полное множество никогда не наблюдается на практике.

2.8.2 RL с верифицируемыми наградами

Обучение с подкреплением оптимизирует стратегию, максимизируя ожидаемую награду. При заданной целевая функция $J(\theta)$ обучение идет через stochastic градиент восхождение или, эквивалентно, через градиентный спуск по отрицательной целевой функции. Поэтому критическое проектное решение — функция награды: она определяет, какие сгенерированные ответы получают положительный сигнал.

В проверяемых областях награда может быть назначена внешней процедурой, не зависящей от субъективного человеческого суждения. Для кода сгенерированную программу можно выполнить на набор тестов; для математики итоговый ответ можно распарсить и сравнить с каноническим решением. В таких случаях событие “ответ является правильным” имеет смысл, внешний по отношению к самой языковой модели. Современные модели рассуждения сильно опираются на эту структуру при обучении [5, 12].

Сначала рассмотрим стандартную постановку с одним эталонным ответом: для каждого запроса x существует ровно одна проверяемая цель y^* . Определим награду

$$R(y, y^*) = \mathbb{I}\{y = y^*\},$$

где $\mathbb{I}\{\cdot\}$ — индикаторная функция. Тогда целевая функция равна

$$J_{\text{RLVR}}(\theta \mid x, y^*) = \mathbb{E}_{z \sim \pi_\theta(z|x)} \mathbb{E}_{y \sim \pi_\theta(y|x,z)} [R(y, y^*)].$$

Эквивалентно, в развернутой форме суммы,

$$J_{\text{RLVR}}(\theta \mid x, y^*) = \sum_{z \in \mathcal{Z}} \sum_{y \in \mathcal{Y}} \pi_{\theta}(z \mid x) \pi_{\theta}(y \mid x, z) R(y, y^*).$$

Для оптимизации этого математического ожидания используется тождество функции оценки, также известная как тождество REINFORCE:

$$\nabla_{\theta} \mathbb{E}_{u \sim p_{\theta}(u)}[f(u)] = \mathbb{E}_{u \sim p_{\theta}(u)}[f(u) \nabla_{\theta} \log p_{\theta}(u)].$$

Это тождество следует напрямую из соотношения $\nabla_{\theta} p_{\theta}(u) = p_{\theta}(u) \nabla_{\theta} \log p_{\theta}(u)$. Применяя его к совместному распределению (z, y) , получаем

$$\begin{aligned} \nabla_{\theta} J_{\text{RLVR}} &= \nabla_{\theta} \sum_{z, y} \pi_{\theta}(z, y \mid x) R(y, y^*) \\ &= \sum_{z, y} R(y, y^*) \nabla_{\theta} \pi_{\theta}(z, y \mid x) \\ &= \sum_{z, y} \pi_{\theta}(z, y \mid x) R(y, y^*) \nabla_{\theta} \log \pi_{\theta}(z, y \mid x) \\ &= \mathbb{E}_{z, y} \left[R(y, y^*) \nabla_{\theta} \log \pi_{\theta}(z, y \mid x) \right]. \end{aligned}$$

Используя факторизацию совместной стратегии,

$$\log \pi_{\theta}(z, y \mid x) = \log \pi_{\theta}(z \mid x) + \log \pi_{\theta}(y \mid x, z),$$

следовательно,

$$\nabla_{\theta} \log \pi_{\theta}(z, y \mid x) = \nabla_{\theta} \log \pi_{\theta}(z \mid x) + \nabla_{\theta} \log \pi_{\theta}(y \mid x, z).$$

Поэтому

$$\nabla_{\theta} J_{\text{RLVR}} = \mathbb{E}_{z, y} \left[R(y, y^*) (\nabla_{\theta} \log \pi_{\theta}(z \mid x) + \nabla_{\theta} \log \pi_{\theta}(y \mid x, z)) \right].$$

Это выражение имеет простую интерпретацию. Выбранная пара (z, y) получает награду только тогда, когда верификатор объявляет y правильным. Когда это происходит, обновление увеличивает и вероятность траектория рассуждения, приведшего к ответу, и вероятность самого ответа при этом траектория.

2.8.3 VeriFree

Юмор не предоставляет такого внешнего верификатора, и в более общем виде многие задачи открытой генерации не имеют детерминированного теста правильности. VeriFree закрывает этот разрыв, замечая, что в случае одного эталонного ответа ожидаемый точное совпадение награда можно переписать как условная вероятность [26].

Зафиксируем x и z . Тогда

$$\begin{aligned}\mathbb{E}_{y \sim \pi_\theta(\cdot | x, z)} [\mathbb{I}\{y = y^*\}] &= \sum_{y \in \mathcal{Y}} \pi_\theta(y | x, z) \mathbb{I}\{y = y^*\} \\ &= \pi_\theta(y^* | x, z).\end{aligned}$$

Подставляя это тождество обратно в RLVR целевая функция, получаем

$$J_{\text{RLVR}}(\theta | x, y^*) = \mathbb{E}_{z \sim \pi_\theta(z | x)} [\pi_\theta(y^* | x, z)].$$

Это мотивирует награду без внешнего верификатора

$$r_\theta(x, z, y^*) := \pi_\theta(y^* | x, z),$$

и соответствующую целевая функция

$$J_{\text{VF}}(\theta | x, y^*) := \mathbb{E}_{z \sim \pi_\theta(z | x)} [r_\theta(x, z, y^*)].$$

При с одним эталонным ответом предположение

$$J_{\text{VF}}(\theta | x, y^*) \equiv J_{\text{RLVR}}(\theta | x, y^*)$$

как скалярные целевые функции. Значимость такой переформулировки одновременно практическая и концептуальная: для вычисления награды не требуется явное декодирование y , а $\pi_\theta(y^* | x, z)$ можно вычислить с помощью принудительного учительского декодирования на последовательность эталонного ответа, что удобно параллелизуется.

Чтобы получить стратегия градиент, запишем

$$J_{\text{VF}}(\theta \mid x, y^*) = \sum_{z \in \mathcal{Z}} \pi_{\theta}(z \mid x) r_{\theta}(x, z, y^*).$$

Дифференцируя по правилу произведения,

$$\begin{aligned} \nabla_{\theta} J_{\text{VF}} &= \sum_z \left[\nabla_{\theta} \pi_{\theta}(z \mid x) r_{\theta}(x, z, y^*) + \pi_{\theta}(z \mid x) \nabla_{\theta} r_{\theta}(x, z, y^*) \right] \\ &= \sum_z \pi_{\theta}(z \mid x) \left[r_{\theta}(x, z, y^*) \nabla_{\theta} \log \pi_{\theta}(z \mid x) + \nabla_{\theta} r_{\theta}(x, z, y^*) \right]. \end{aligned}$$

Так как $r_{\theta}(x, z, y^*) = \pi_{\theta}(y^* \mid x, z)$,

$$\nabla_{\theta} r_{\theta}(x, z, y^*) = \pi_{\theta}(y^* \mid x, z) \nabla_{\theta} \log \pi_{\theta}(y^* \mid x, z) = r_{\theta}(x, z, y^*) \nabla_{\theta} \log \pi_{\theta}(y^* \mid x, z).$$

Следовательно,

$$\nabla_{\theta} J_{\text{VF}} = \mathbb{E}_{z \sim \pi_{\theta}(z \mid x)} \left[r_{\theta}(x, z, y^*) (\nabla_{\theta} \log \pi_{\theta}(z \mid x) + \nabla_{\theta} \log \pi_{\theta}(y^* \mid x, z)) \right].$$

Эта декомпозиция информативна. Первый член,

$$\mathbb{E}_z \left[r_{\theta}(x, z, y^*) \nabla_{\theta} \log \pi_{\theta}(z \mid x) \right],$$

является слагаемое функции оценки по выбранный траектории рассуждения. Он повышает вероятность траектории, при которых сама модель назначает более высокую условный масса эталонный ответ. Второй член,

$$\mathbb{E}_z \left[\nabla_{\theta} r_{\theta}(x, z, y^*) \right],$$

является прямой teacher-forced градиент на последовательность эталонного ответа. По сравнению с RLVR, VeriFree убирает шум от выборки итогового ответа y : оценитель все еще выборки z , но не ответ, используемый для оценка награды. Именно поэтому VeriFree привлекателен в доменах, где верификатор недоступен, но один эталонный ответ существует.

2.8.4 С несколькими эталонными ответами VeriFree

Один-эталонный ответ предположение не подходит для юмора. Запрос вроде “напиши шутка используя ключевые слова cat, dog” не имеет одного правильного продолжение; он имеет много правдоподобных продолжений, различающихся завязка, механизм панчлайна, тон и уровнем абсурдности. Фактически реализованный репозиторий явно предназначен для построения нескольких эталонных ответов для одного слабого запроса. Поэтому награда должна быть определен на множестве допустимых ответах, а не на одном ответе.

Пусть $Y^*(x) \subset \mathcal{Y}$ обозначает наблюдаемый набор допустимый эталонные ответы для запроса x . Определим проверяемую награду с несколькими эталонными ответами

$$R(y, Y^*) = \mathbb{I}\{y \in Y^*\}.$$

Соответствующая set-valued точное совпадение целевая функция равна

$$J_{\text{MR}}(\theta \mid x, Y^*) = \mathbb{E}_{z \sim \pi_\theta(z|x)} \mathbb{E}_{y \sim \pi_\theta(\cdot|x, z)} [\mathbb{I}\{y \in Y^*\}].$$

Для фиксированных x и z ,

$$\begin{aligned} \mathbb{E}_{y \sim \pi_\theta(\cdot|x, z)} [\mathbb{I}\{y \in Y^*\}] &= \sum_{y \in \mathcal{Y}} \pi_\theta(y \mid x, z) \mathbb{I}\{y \in Y^*\} \\ &= \sum_{y \in Y^*} \pi_\theta(y \mid x, z). \end{aligned}$$

Это мотивирует с несколькими эталонными ответами награду без внешнего верификатора

$$r_\theta(x, z, Y^*) := \sum_{y \in Y^*} \pi_\theta(y \mid x, z).$$

Величина $r_\theta(x, z, Y^*)$ лежит в $[0, 1]$: это total вероятностная масса, которую стратегия назначает, при траектория рассуждения z , наблюдаемый набор эталонных ответов. Эквивалентно, это вероятность события, что сгенерированный ответ попадает внутрь этого множества. Итоговая целевая функция равна

$$J_{\text{MRVF}}(\theta \mid x, Y^*) = \mathbb{E}_{z \sim \pi_\theta(z|x)} [r_\theta(x, z, Y^*)].$$

На уровне скалярной целевой функции MRVF играет для set-valued награ-

да ту же роль, что VeriFree играет для single точное совпадение награда. Однако градиент structure не является просто с одним эталонным ответом формулой, где y^* заменено символом множества. Со стороны ответа производная меняется существенно.

Действительно,

$$J_{\text{MRVF}}(\theta \mid x, Y^*) = \sum_z \pi_\theta(z \mid x) r_\theta(x, z, Y^*),$$

поэтому

$$\begin{aligned} \nabla_\theta J_{\text{MRVF}} &= \sum_z \left[\nabla_\theta \pi_\theta(z \mid x) r_\theta(x, z, Y^*) + \pi_\theta(z \mid x) \nabla_\theta r_\theta(x, z, Y^*) \right] \\ &= \mathbb{E}_{z \sim \pi_\theta(z \mid x)} \left[r_\theta(x, z, Y^*) \nabla_\theta \log \pi_\theta(z \mid x) + \nabla_\theta r_\theta(x, z, Y^*) \right]. \end{aligned}$$

Теперь раскроем по эталонным ответам производная:

$$\begin{aligned} \nabla_\theta r_\theta(x, z, Y^*) &= \nabla_\theta \sum_{y \in Y^*} \pi_\theta(y \mid x, z) \\ &= \sum_{y \in Y^*} \nabla_\theta \pi_\theta(y \mid x, z) \\ &= \sum_{y \in Y^*} \pi_\theta(y \mid x, z) \nabla_\theta \log \pi_\theta(y \mid x, z). \end{aligned}$$

Поэтому

$$\nabla_\theta J_{\text{MRVF}} = \mathbb{E}_{z \sim \pi_\theta(z \mid x)} \left[r_\theta(x, z, Y^*) \nabla_\theta \log \pi_\theta(z \mid x) + \sum_{y \in Y^*} \pi_\theta(y \mid x, z) \nabla_\theta \log \pi_\theta(y \mid x, z) \right].$$

Иногда полезно переписать второй член в нормализованной форме. Если $r_\theta(x, z, Y^*) > 0$, определим

$$w_\theta(y \mid x, z, Y^*) := \frac{\pi_\theta(y \mid x, z)}{r_\theta(x, z, Y^*)} \mathbb{I}\{y \in Y^*\}.$$

Тогда $w_\theta(\cdot \mid x, z, Y^*)$ является вероятностное распределение over наблюдаемый

набор эталонных ответов, и

$$\nabla_{\theta} r_{\theta}(x, z, Y^*) = r_{\theta}(x, z, Y^*) \sum_{y \in Y^*} w_{\theta}(y \mid x, z, Y^*) \nabla_{\theta} \log \pi_{\theta}(y \mid x, z).$$

Эквивалентно,

$$\nabla_{\theta} \log r_{\theta}(x, z, Y^*) = \sum_{y \in Y^*} w_{\theta}(y \mid x, z, Y^*) \nabla_{\theta} \log \pi_{\theta}(y \mid x, z),$$

и, следовательно,

$$\nabla_{\theta} J_{\text{MRVF}} = \mathbb{E}_{z \sim \pi_{\theta}(z \mid x)} \left[r_{\theta}(x, z, Y^*) \left(\nabla_{\theta} \log \pi_{\theta}(z \mid x) + \nabla_{\theta} \log r_{\theta}(x, z, Y^*) \right) \right].$$

Эта форма проясняет отличие от с одним эталонным ответом VeriFree. В VeriFree прямой term — это градиент $\log \pi_{\theta}(y^* \mid x, z)$ для одного эталонного ответа. В MRVF прямой term — это градиент $\log \sum_{y \in Y^*} \pi_{\theta}(y \mid x, z)$. Поэтому обновление не отдает априорного предпочтения одному эталонный ответ; он увеличивает полная вероятностная масса на наблюдаемый набор, причем каждый эталонный ответ взвешивается текущей вероятностной массе стратегии, назначенной ему. Если один эталонный ответ уже доминирует, градиент концентрируется на нем. Если масса распределена по нескольким эталонные ответы, градиент усиливает их совместно. Поэтому MRVF не является просто нотационным вариантом VeriFree: оптимизация геометрия меняется с одной целевая строка на логарифм суммы по многим целевым строкам.

2.8.5 Неполные наблюдаемые эталонные ответы

Наблюдаемый set $Y^*(x)$ все еще не является полным множеством хороших шуток. Возникают две разные формы неполнота.

Структурная неполнота. Даже если репозиторий извлекает несколько эталонные ответы для запрос, может существовать много дополнительных шуток в скрытое множество $Y_{\text{true}}(x)$, которые также валидны, но не наблюдаются. Это неизбежно в творческий области. Конечный набор данных не может перечислить все допустимые панчлайны для слабый запрос.

Вычислительная неполнота. Даже внутри наблюдаемый набор $Y^*(x)$ вычисление

$$r_\theta(x, z, Y^*) = \sum_{y \in Y^*} \pi_\theta(y \mid x, z)$$

может стать дорогим, когда $|Y^*|$ велико. Поэтому сумму можно оценивать на выбранный subset $Y \subseteq Y^*$.

Предположим, что Y выбранный uniformly without замена из Y^* , и каждый эталонный ответ $y \in Y^*$ имеет вероятность включения

$$\rho = P(y \in Y) = \frac{|Y|}{|Y^*|}.$$

Определим subset награда

$$r_\theta(x, z, Y) := \sum_{y \in Y} \pi_\theta(y \mid x, z) = \sum_{y \in Y^*} \mathbb{I}\{y \in Y\} \pi_\theta(y \mid x, z).$$

Беря математическое ожидание по случайному подмножеству,

$$\begin{aligned} \mathbb{E}_Y [r_\theta(x, z, Y)] &= \sum_{y \in Y^*} \mathbb{E}_Y [\mathbb{I}\{y \in Y\}] \pi_\theta(y \mid x, z) \\ &= \sum_{y \in Y^*} P(y \in Y) \pi_\theta(y \mid x, z) \\ &= \rho \sum_{y \in Y^*} \pi_\theta(y \mid x, z) \\ &= \rho r_\theta(x, z, Y^*). \end{aligned}$$

Следовательно, subset награда пропорционален в математическое ожидание полный по наблюдаемому набору награда.

Здесь становится релевантным использование базового уровня в GRPO. Предположим, что для фиксированного запрос x все выбранные награды в группе умножаются на один и тот же положительный множитель ρ_x , например потому что используется только доля наблюдаемые эталонные ответы. Если стандартизовать награды внутри группы,

$$\tilde{A}_i = \frac{\hat{r}_i - \mu_x}{\sigma_x + \varepsilon}, \quad \mu_x = \frac{1}{G} \sum_{j=1}^G \hat{r}_j,$$

то умножение каждого $\hat{r}_i$ на $\rho_x > 0$ умножает и числитель, и знаменатель на один и тот же множитель, оставляя $\tilde{A}_i$ неизменным. В этом смысле GRPO группа standardization масштаб-invariant. Она смягчает сдвиги масштаба награды в слагаемое функции оценки, хотя не восстанавливает информацию, потерянную из-за пропущенные эталонные ответы, и сама по себе не решает смещение прямого градиента по эталонным ответам.

Другая фундаментальная проблема — проблема ложных отрицательных примеров. Оптимизация

$$r_\theta(x, z, Y^*) = \sum_{y \in Y^*} \pi_\theta(y \mid x, z)$$

увеличивает вероятностная масса на наблюдаемые эталонные ответы. Поскольку условное распределение по всем строкам суммируется в единицу,

$$\sum_{y \in \mathcal{Y}} \pi_\theta(y \mid x, z) = 1,$$

увеличенная масса должна прийти из $\mathcal{Y} \setminus Y^*$. К сожалению, это дополнение содержит не только плохие шутки, но и ненаблюдаемые хорошие шутки из $Y_{\text{true}}(x) \setminus Y^*(x)$. Иными словами, обучение signal может непреднамеренно подавлять валидные, но ненаблюдаемые удачные юмористические продолжения. Этот риск не является дефектом градиент вывод; он является структурным следствием обучение from неполная разметка в область с естественно большим числом допустимых решений.

Чтобы уменьшить этот риск, предлагаемая оптимизация включает штраф Кульбака–Лейблера относительно опорной стратегии π_{ref} :

$$\max_{\theta} \mathbb{E}_{x,z} [r_\theta(x, z, Y^*(x))] - \beta \mathbb{E}_x [\text{KL}(\pi_\theta(\cdot \mid x) \parallel \pi_{\text{ref}}(\cdot \mid x))].$$

Эквивалентно, можно рассматривать задачу в форме задачи с ограничением,

$$\max_{\theta} \mathbb{E}_{x,z} [r_\theta(x, z, Y^*(x))] \quad \text{s.t.} \quad \mathbb{E}_x [\text{KL}(\pi_\theta(\cdot \mid x) \parallel \pi_{\text{ref}}(\cdot \mid x))] \leq \varepsilon.$$

В данном контексте KL-слагаемое не является просто обычной регуляризацией. Его методологическая роль — препятствовать чрезмерно агрессивному перераспределению вероятностной массы от опорной модели, когда наблюдаемые

наборы эталонных ответов заведомо неполны.

2.8.6 Предлагаемый цикл обучения

Предполагаемый этап обучения выборки несколько траектории рассуждения для одного запроса и использует их награды для обновления стратегия рассуждения. Для запроса x выборка группа

$$z_1, \dots, z_G \sim \pi_\theta(z \mid x).$$

Для каждого z_i вычисляется награда по нескольким эталонным ответам

$$\hat{r}_i := r_\theta(x, z_i, Y^*(x)) = \sum_{y \in Y^*(x)} \pi_\theta(y \mid x, z_i).$$

На практике каждая $\pi_\theta(y \mid x, z_i)$ получается с помощью принудительного учительского декодирования на последовательность эталонного ответа y . Поскольку эти вероятности последовательностей очень малы, реализация должна аккумулировать логарифмы вероятностей токенов и, где необходимо, использовать стабилизацию вроде $\log \sum \exp$ перед обратным переходом к вероятностям или нормализованным весам.

Градиент естественно раскладывается на три части:

$$\nabla_\theta J_{\text{MRVF}} = g_z + g_r + g_{\text{KL}}.$$

Рассуждение term. Функция оценки часть по z оценивается по выбранный траектории и поэтому выигрывает от снижение дисперсии. Чистый выбор — с исключением текущего элемента базовая модель

$$b_i = \frac{1}{G-1} \sum_{j \neq i} \hat{r}_j, \quad A_i = \hat{r}_i - b_i.$$

Соответствующий оценитель равен

$$g_z = \frac{1}{G} \sum_{i=1}^G A_i \nabla_\theta \log \pi_\theta(z_i \mid x).$$

Это та же логика, которая мотивирует RLOO оценители: каждая траектория сравнивается с другими траектории, выбранный для того же запроса. При желании можно дополнительно нормализовать A_i внутри группы, чтобы уменьшить запрос-specific изменчивость масштаба награды, в духе GRPO. Такая нормализация повышает устойчивость, но уже не является строго несмещенной, поскольку масштаб оценка сама зависит от выбранный группа.

Эталонный ответ term. Для прямой ответ-side вклад корректный градиент по наблюдаемому набору равен

$$g_r = \frac{1}{G} \sum_{i=1}^G \nabla_{\theta} r_{\theta}(x, z_i, Y^*(x)) = \frac{1}{G} \sum_{i=1}^G \sum_{y \in Y^*(x)} \pi_{\theta}(y \mid x, z_i) \nabla_{\theta} \log \pi_{\theta}(y \mid x, z_i).$$

Эквивалентно, используя нормализованные веса внутри набора

$$w_{i,y} := \frac{\pi_{\theta}(y \mid x, z_i)}{\hat{r}_i} \mathbb{I}\{y \in Y^*(x)\},$$

можно записать

$$g_r = \frac{1}{G} \sum_{i=1}^G \hat{r}_i \sum_{y \in Y^*(x)} w_{i,y} \nabla_{\theta} \log \pi_{\theta}(y \mid x, z_i),$$

если $\hat{r}_i > 0$. Эта форма делает геометрия явной: каждая траектория получает total weight $\hat{r}_i$, а внутри этого траектория градиент распределяется по эталонным ответам согласно текущей вероятностной массе стратегии на каждом эталонный ответ.

KL-слагаемое. Наконец,

$$g_{\text{KL}} = -\beta \nabla_{\theta} \text{KL}(\pi_{\theta}(\cdot \mid x) \parallel \pi_{\text{ref}}(\cdot \mid x)).$$

Полный обновление для мини-пакет усредняет эти запрос-уровень вклады по выбранный запросы.

Практическая важность реализованного процесса теперь ясна. Без набора, обусловленные запросом с несколькими эталонными ответами $Y^*(x)$ нельзя вычислить ни $\hat{r}_i$, ни g_r . Поэтому процесс подготовки данных не является вспо-

могательным к теории; он предоставляет наблюдаемый structure, от которой зависит MRVF оптимизация.

2.8.7 Замечание о использовании базового уровня

Дисперсия снижение прямолинейно применима только к слагаемое функции оценки. Причина в том, что z действительно выбранный from $\pi_\theta(z \mid x)$. Если $b(x, z_{-i})$ — базовая модель, не зависящий от выбранный variable z_i , то

$$\begin{aligned}\mathbb{E}_{z_i \sim \pi_\theta(\cdot \mid x)}[b(x, z_{-i}) \nabla_\theta \log \pi_\theta(z_i \mid x)] &= b(x, z_{-i}) \sum_{z_i} \pi_\theta(z_i \mid x) \nabla_\theta \log \pi_\theta(z_i \mid x) \\ &= b(x, z_{-i}) \sum_{z_i} \nabla_\theta \pi_\theta(z_i \mid x) \\ &= b(x, z_{-i}) \nabla_\theta 1 \\ &= 0.\end{aligned}$$

Поэтому с исключением текущего элемента базовая модель несмещен для слагаемого рассуждения.

Эталонный ответ term устроен иначе. Это не математическое ожидание по выбранным ответам $y \sim \pi_\theta(\cdot \mid x, z)$, а прямой производная summed вероятность эталонного ответа. Следовательно, нет тождества вида

$$\mathbb{E}_y[\nabla_\theta \log \pi_\theta(y \mid x, z)] = 0$$

доступного для центрирования. Например, в случае одного эталонного ответа,

$$\mathbb{E}_z[(r_\theta(x, z, y^*) - b(x)) \nabla_\theta \log \pi_\theta(y^* \mid x, z)]$$

равно

$$\mathbb{E}_z[r_\theta(x, z, y^*) \nabla_\theta \log \pi_\theta(y^* \mid x, z)] - b(x) \mathbb{E}_z[\nabla_\theta \log \pi_\theta(y^* \mid x, z)],$$

и второй член в общем случае не равен нулю. Та же проблема сохраняется в с несколькими эталонными ответами случай с $\nabla_\theta \log r_\theta(x, z, Y^*)$. Поэтому базовые модели, безвредные для слагаемое функции оценки, становятся смещенными, если наивно применить их к прямой градиент по эталонным ответам.

Это различие важно для дизайна обучения алгоритм. Рассуждение term должен использовать variance-снижение механизмы, такую как с исключением текущего элемента базовые модели или аккуратно выбранную группа нормализация. Эталонный ответ term, напротив, следует вычислять максимально прямо из по наблюдаемому набору вероятность, принимая, что это детерминированный, но потенциально смещенный приближенная мера для истинный награда, порождаемого $Y_{\text{true}}(x)$.

В совокупности эти наблюдения объясняют общую структуру предложения. MRVF наследует ключевое вычислительное преимущество обучения без внешнего верификатора: он избегает внешние верификаторы и явный награда выборка over итоговые ответы, но должен быть адаптирован к с несколькими эталонными ответами и неполному характеру юмора. Итоговая целевая функция математически вычислима, но ее ограничения являются частью вклада работы: неполнота, ложные отрицательные примеры и построение оценителя центральны для проблемы.

Глава 3

Результаты

3.1 Текущие результаты

Реализованный процесс уже поддерживает сбор корпуса, генерацию векторных представлений, извлечение ключевых слов, построение запросы, поиск эталонных ответов, генерацию кандидатов и автоматическая попарная оценка. Оставшийся отсутствующий этап — собственно обучения с подкреплением обновление цикл. Поэтому данный раздел является узким: он описывает завершенные артефакты данных и фиксирует конкретный экспериментальный постановка для первых MRVF обучающие запуски.

Результат на уровне корпуса — объединенная коллекция шуток примерно из 664k объектов. В финальной схема обучения этот полный корпус служит пул поиска: все шутки кодируются векторных представлений и индексируются, а эталонные ответы для слабые запросы извлекаются из этого полного пространства векторных представлений. Однако извлечение ключевых слов нужно применять только к достаточно большому подмножеству, чтобы достичь целевого объем выборки. В нашем случае 3 — 5k группы ключевых слов могут дать 10 — 20k эталонные ответы для обучение, валидация и тестирование.

Для предварительный этап этого достаточно. Важный результат состоит в том, что данные и экспериментальный постановка теперь финализированы: корпус существует, эталонный ответ коллекция реализована, методология зафиксирована, а оставшаяся работа ограничена цикл обучения MRVF.

3.2 Планируемая процедура обучения

Репозиторий уже реализует полный процесс подготовки данных на Python с использованием Hugging Face Наборы данных, Parquet артефакты, FAISS-

based ближайших соседей поиск и OpenAI-compatible API вызовы для векторное представление генерация, генерацию кандидатов и автоматический оценка. Однако для собственно обучающие запуски стек программного обеспечения немного изменится.

Планируемая реализация будет использовать модифицированную версию **trl** для оптимизация и **vLLM** для генерация. **trl** предоставляет со стороны обучения логика для оптимизация стратегии, тогда как **vLLM** будет использоваться и для генерации ответы базовых моделей, и для обслуживания траектории генерации во время обучение с подкреплением. Поскольку MRVF зависит от выборка нескольких траектории рассуждения на запрос, пропускная способность генерации траекторий важен для эффективности.

Целевые модели относятся к семейству Qwen3 в диапазоне размеров 1.7B и 4B. Для каждого размера планируются следующие базовые модели:

1. **Qwen3 Instruct**, базовая модель в без режима рассуждения режим;
2. **Qwen3 Thinking**, базовая модель в thinking режим;
3. **MRVF-обученных Qwen3 Thinking**, предлагаемая модель;

Такая постановка сохраняет интерпретируемость сравнения. Он изолирует три отдельных вопроса: достаточно ли обычной инструкция настройка для генерации юмора; улучшает ли родной thinking режим генерацию юмора уже без дополнительного обучения; и может ли MRVF дополнительно улучшить thinking модель.

Все обучающие запуски планируются на 1–2 NVIDIA H200 GPUs в HSE HPC кластер. Этого достаточно для целевых размеров моделей и позволяет распараллеливать эксперименты между объекты.

Сравнение будет использовать процесс оценки, уже реализованный в репозитории. Ответы сопоставляются для одного запроса, порядок левого и правого вариантов рандомизируется, оценщик принуждается выбрать ровно одного победитель, а оценки моделей агрегируются через Bradley-Terry рейтинг. Этого достаточно для эксперимента, поскольку цель состоит в обнаружении того, дает ли MRVF измеримый сдвиг относительно instruct и базовые модели в режиме рассуждения. Более крупная многомерный оценка с оценивание по рубрике и человеческую разметку планируется после завершения обучения.

3.3 Планируемые абляции

План абляций напрямую связан с теорией. Наиболее информативные абляции следующие:

- **Число наблюдаемых эталонных ответов на запрос $|Y^*|$.** Сравнить меньшие и большие наборы эталонных ответов, например через обрезку результатов поиска до 1, 3 или 5 объектов.
- **коэффициент $KL \beta$.** Проверить, насколько сильно модель должна быть удерживаться рядом с начальной контрольной точкой, чтобы компенсировать неполнота.
- **Размер группы G .** Варьировать число выбранный траектории рассуждения на один запрос, чтобы оценить variance / пропускная способность компромисс MRVF оценщик.
- **Размер модели.** Сравнить 1.7B и 4B, а также проверить, насколько MRVF устойчив при применении к меньшим моделям.

Заключение

Данная работа рассматривала генерацию юмора как задачу рассуждения в неверифицируемом домене. Центральное утверждение состояло в том, что обучение юмору должно отходить от имитации одного референса и переходить к распределению вероятностной массы по множеству эталонных ответов при слабых запросах. Обзор литературы показал, почему такой сдвиг необходим: SFT склонен к имитации и запоминанию, основанный на предпочтениях RL нестабилен в юморе из-за шумных оценок, а современные многошаговые системы генерации юмора повышают качество ценой жестко заданных структур рассуждения.

Работа внесла два связанных вклада. На методологическом уровне она сформулировала расширение обучения с подкреплением без внешнего верификатора на случай нескольких эталонных ответов, в котором награда задается полной вероятностной массой, назначенной множеству приемлемых эталонных ответов, а не одному золотому ответу. На уровне реализации она построила и проанализировала пилотный процесс, который преобразует большой корпус шуток в запрос-эталонный ответ наборы через векторных представлений, извлечение ключевых слов, векторный поиск, удаление дублей и разбиение данных.

Эмпирический анализ пилота показал, что идея является практически осуществимой. В текущем локальном состоянии репозитория уже собран корпус из 664,048 шуток, пилотное подмножество было встроенный и обработано, а итоговый рабочий процесс производит 5,058 дедулицированных запрос-эталонный ответ строк со средним числом эталонных ответов 4.07 на запрос и медианой пять. Эти статистики показывают, что ключевое структурное требование обучения MRVF, а именно наличие запросы с несколькими приемлемыми эталонными ответами, может быть удовлетворено на практике.

В то же время работа остается предварительной в корректном смысле. Процедура построения запросы все еще приближенная, последующий артефакты контрольного набора еще не закоммичены, а сам цикл обучения MRVF пока

не реализован. Поэтому данную работу следует читать как основу дипломного исследования, а не как окончательный эмпирический вердикт об обучении с подкреплением для юмора.

Главный вывод состоит в том, что проект уже перешел границу от спекулятивного отчета к защищаемой исследовательской программе. Представление данных, программный процесс и формальная целевая функция согласованы друг с другом. Следующий шаг состоит в реализации этапа обучения и проверке того, сможет ли построенный набор данных с несколькими эталонными ответами действительно вызывать более качественное рассуждение и более сильный юмор, чем стандартные базовые методы постобучения.

Список литературы

- [1] Mostafa Amin и Manuel Burghardt. «A survey on approaches to computational humor generation». В: *Proceedings of the 4th Joint SIGHUM Workshop on Computational Linguistics for Cultural Heritage, Social Sciences, Humanities and Literature*. 2020, с. 29—41. URL: <https://aclanthology.org/2020.latechclfl-1.4/>.
- [2] Yahui Chen, Buxin Shi и Meng Si. *Prompt to GPT-3: Step-by-step thinking instructions for humor generation*. arXiv:2306.13195. 2023. URL: <https://arxiv.org/abs/2306.13195>.
- [3] Tianyu Chu, Shuhua Zhang, Yu Chen, Ning Ding и Tao Yu. *SFT memorizes, RL generalizes: A comparative study of foundation model post-training*. OpenReview / arXiv preprint. 2025. URL: <https://openreview.net/forum?id=2P4rYTtGok>.
- [4] Andrea Cocchieri, Lorenzo Ragazzi, Pietro Italiani, Giuseppe Tagliavini и Gianluca Moro. «“What do you call a dog that is incontrovertibly true? Dogma”: Testing LLM generalization through humor». В: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2025, с. 22922—22937. URL: <https://doi.org/10.18653/v1/2025.acl-long.1117>.
- [5] DeepSeek-AI. *DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning*. arXiv:2501.12948. 2025. URL: <https://doi.org/10.48550/arXiv.2501.12948>.
- [6] Leo Gao, John Schulman, Jacob Hilton и др. *Scaling laws for reward model overoptimization*. arXiv:2210.10760. 2023. URL: <https://arxiv.org/abs/2210.10760>.
- [7] Akul Gunjal, Alex Wang, Esther Lau, Vedaant Nath, Yichao He, Bo Liu и Sean Hendryx. *Rubrics as rewards: Reinforcement learning beyond verifiable*

- domains*. arXiv:2507.17746. 2025. URL: <https://doi.org/10.48550/arXiv.2507.17746>.
- [8] Hamid Hashemi, Jason Eisner, Corby Rosset, Benjamin Van Durme и Chris Kedzie. «LLM-Rubric: A multidimensional, calibrated approach to automated evaluation of natural language texts». B: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*. 2024. URL: <https://aclanthology.org/2024.acl-long.745/>.
 - [9] Justine T. Kao, Roger Levy и Noah D. Goodman. «A computational model of linguistic humor in puns». B: *Cognitive Science* 40.5 (2016), с. 1270—1285. URL: <https://doi.org/10.1111/cogs.12269>.
 - [10] Sangho Kim и Lydia B. Chilton. *AI humor generation: Cognitive, social and creative skills for effective humor*. arXiv:2502.07981. 2025. URL: <https://arxiv.org/abs/2502.07981>.
 - [11] Jens Lemmens и Victor De Marez. «Computational humor modeling: A survey on the state of the art». B: *ACM Computing Surveys* 58.7 (2026), 177:1—177:37. URL: <https://www.uantwerpen.be/en/staff/jens-lemmens/publications/>.
 - [12] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever и Karl Cobbe. *Let’s verify step by step*. OpenAI / arXiv preprint. 2023. URL: <https://arxiv.org/abs/2305.20050>.
 - [13] Thomas Loakman, William Thorne и Chien-Sheng Lin. «Comparing apples to oranges: A dataset & analysis of LLM humour understanding from traditional puns to topical jokes». B: *Findings of the Association for Computational Linguistics: EMNLP 2025*. 2025, с. 9502—9518. URL: <https://doi.org/10.18653/v1/2025.findings-emnlp.505>.
 - [14] Thomas Loakman, William Thorne и Chien-Sheng Lin. *Who’s laughing now? An overview of computational humour generation and explanation*. arXiv:2509.21175. 2025. URL: <https://arxiv.org/abs/2509.21175>.
 - [15] Graeme Ritchie. «Current directions in computational humour». B: *Artificial Intelligence Review* 16.2 (2001), с. 119—135. URL: <https://doi.org/10.1023/A:1011610210506>.

- [16] Ryo Sakabe, Hwiyeon Kim, Tatsuya Hirasawa и Mamoru Komachi. *Assessing the capabilities of LLMs in humor: A multi-dimensional analysis of Oogiri generation and evaluation*. arXiv:2511.09133. 2025. URL: <https://arxiv.org/abs/2511.09133>.
- [17] Alexey Tikhonov и Pavel Shtykovskiy. *Humor mechanics: Advancing humor generation with multistep reasoning*. arXiv:2405.10073. 2024. URL: <https://arxiv.org/abs/2405.10073>.
- [18] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc Le и Denny Zhou. *Chain-of-thought prompting elicits reasoning in large language models*. arXiv:2201.11903. 2022. URL: <https://doi.org/10.48550/arXiv.2201.11903>.
- [19] Orion Weller и Kevin Seppi. «The rJokes dataset: A large scale humor collection». B: *Proceedings of the Twelfth Language Resources and Evaluation Conference*. 2020, с. 6136—6141. URL: <https://aclanthology.org/2020.lrec-1.753/>.
- [20] Tim Winters и Sam Van der Stockt. «Evaluating humor generation in an improvisational comedy setting». B: *Computational Linguistics in the Netherlands Journal* 14 (2025), с. 505—523. URL: <https://www.clinjournal.org/clinj/article/view/214>.
- [21] Zhen Xu, Shaojie Yuan, Li Chen и Diyi Yang. «“A good pun is its own reword”: Can large language models understand puns?» B: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 2024, с. 11766—11782. URL: <https://doi.org/10.18653/v1/2024.emnlp-main.657>.
- [22] Tianyu Yu, Bin Ji, Shun Wang, Shunyu Yao, Zhijian Wang, Ganqu Cui, Luqi Yuan, Ning Ding, Yong Yao, Zhiyuan Liu, Maosong Sun и Tat-Seng Chua. *RLPR: Extrapolating RLVR to general domains without verifiers*. arXiv:2506.18254. 2025. URL: <https://doi.org/10.48550/arXiv.2506.18254>.
- [23] Jiawei Zhang, Sizhe Luo, Ruijie Zhang и Qi Su. *HUMORCHAIN: Theory-guided multi-stage reasoning for interpretable multimodal humor generation*. arXiv:2511.21732. 2025. URL: <https://doi.org/10.48550/arXiv.2511.21732>.

- [24] Jie Zhang, Lily Jain, Yichi Guo, Justin Chen, Kexin Linda Zhou, Sreeranjani Suresh, Amanda Wagenmaker, Scott Sievert, Taylor Rogers, Kevin Jamieson, Robert Mankoff и Robert Nowak. *Humor in AI: Massive scale crowd-sourced preferences and benchmarks for cartoon captioning*. Advances in Neural Information Processing Systems, Datasets and Benchmarks Track. 2024. URL: <https://arxiv.org/abs/2406.10522>.
- [25] Shaoxiong Zhong, Zicheng Huang, Shufan Gao, Wen Wen, Lin Lin, Marinka Zitnik и Pan Zhou. «Let’s think outside the box: Exploring leap-of-thought in large language models with creative humor generation». B: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2024, c. 13246—13257. URL: https://openaccess.thecvf.com/content/CVPR2024/html/Zhong_Lets_Think_Outside_the_Box_Exploring_Leap-of-Thought_in_Large_Language_CVPR_2024_paper.html.
- [26] Xingcheng Zhou, Zihan Liu, Ariel Sims, Han Wang, Tianyu Pang, Chao Li, Li Wang, Min Lin и Chao Du. *Reinforcing general reasoning without verifiers*. arXiv:2505.21493. 2025. URL: <https://arxiv.org/abs/2505.21493>.

Приложение А

Реализованные шаблоны запросов

В этом приложении фиксируются основные шаблоны, уже реализованные в репозитории. Они включены сюда, поскольку задают операциональную форму запросы, используемых на протяжении пилотного эксперимента.

А.1 Запрос для генерации эталонных ответов

Запрос, используемый для поиска эталонных ответов и генерацию кандидатов:

Напиши шутка используя следующих ключевое слово(s): {ключевые слова}

Его главное преимущество — согласованность. Одна и та же текстовая форма используется для запроса для векторного представления во время поиска и для последующей генерации кандидатов, что уменьшает несоответствие между построением набора данных и оценкой. Главный недостаток состоит в том, что шаблон намеренно общий и поэтому не кодирует ни теорию юмора, ни информацию об аудитории.

А.2 Инструкции для запросов для векторного поиска

Для адаптации моделью векторных представлений к поиску подзадачи используются два коротких инструкционных шаблона:

Instruct: Given ключевое слово для шутки, извлекать шутки
would совпадение ключевое слово.
Query: {ключевое слово}

Instruct: Given запрос для шутка, извлекать шутки что
would ответ запрос.
Query: {запрос}

Эти шаблоны важны, потому что пилотный не использует исходные строки ключевых слов как векторных представлений напрямую. Он кодирует их внутри инструкции, ориентированные на поиск, что делает векторное пространство семантического намерения более специализированным под задачу.

А.3 Оценочный запрос

Системная инструкция для попарная оценка просит оценщик сравнить две шутки, написанные для одного запроса, и выбрать ровно одного победитель. Инструкция явно не поощряет вознаграждать длину или стилистическую выверенность, если они не улучшают юмор. Такой дизайн намеренно прост и должен рассматриваться как базовая модель протокол оценки, а не как финальная рубрика, согласованная с человеческими оценками.

Приложение В

Дополнительные замечания о программной архитектуре

Репозиторий организован как набор асинхронный процессы:

- `JokesPipeline`: скачивает и предобрабатывает сырые корпуса;
- `EmbeddingsPipeline`: вычисляет плотные векторные представления шуток;
- `KeywordsPipeline`: извлекает значимые ключевые слова через сходство и MMR;
- `ReferencesPipeline`: строит наборы данных вида «запрос — эталонные ответы» через векторный поиск;
- `CandidatesPipeline`: генерирует шутки по запросам;
- `EvaluationPipeline`: выполняет попарное оценивание и агрегацию рейтинга.

Эта архитектура важна для финального диплома, потому что она отделяет построение данных от сравнения моделей. В результате последующий этап обучения MRVF можно добавить без переписывания остальной инфраструктуры. Отсутствующий компонент — именно тот, который теперь становится основной целью будущей работы: процесс обучения, который потребляет построенный запрос-набор эталонных ответов и оптимизирует формальную целевую функцию, введенную в Chapter 2.